

Nexus Claims GenAI - Solution Architecture

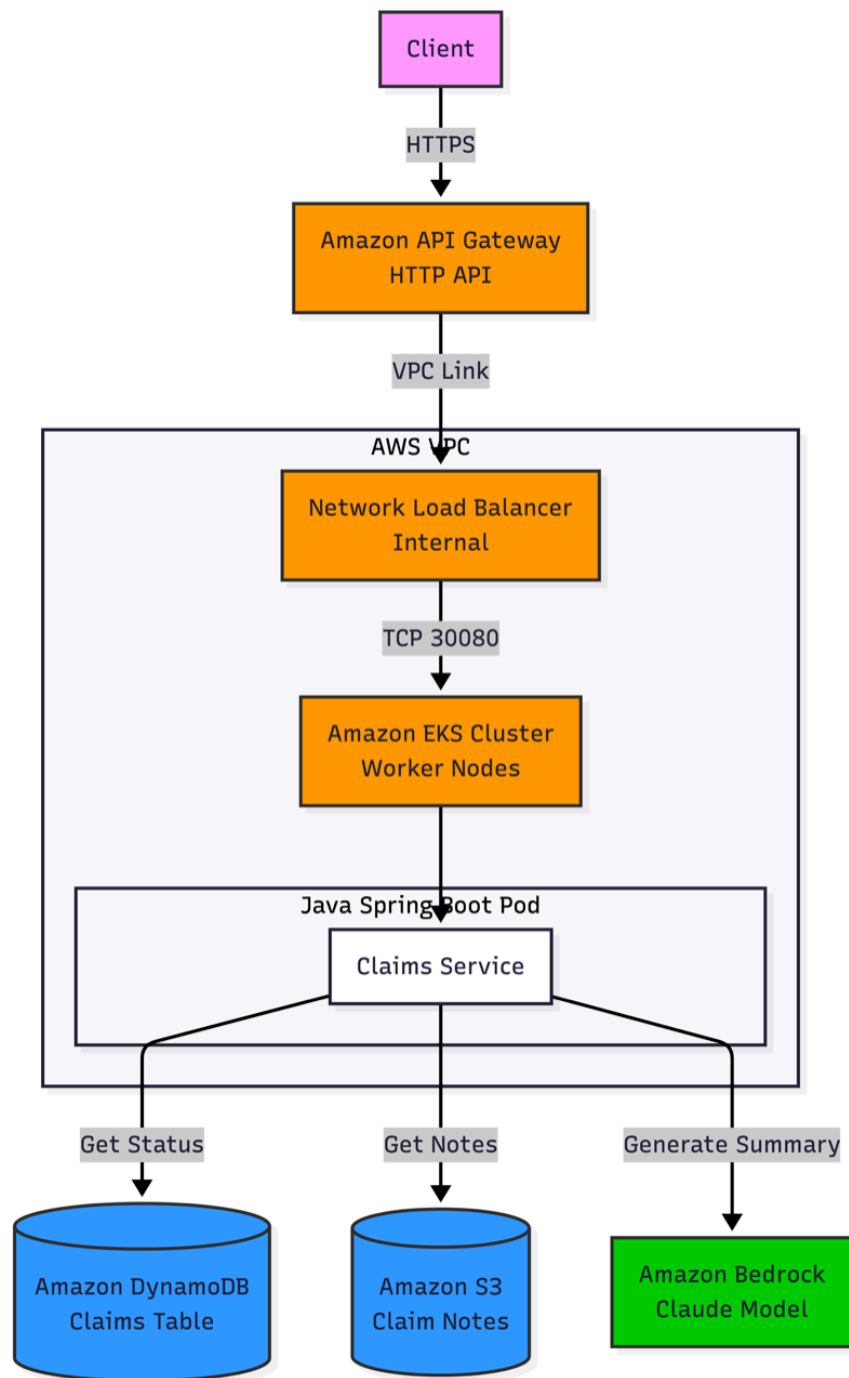
1. Executive Summary

The Challenge: Insurance claim processing faces a critical bottleneck: the manual review of unstructured adjusters' notes. Adjusters spend valuable time reading through lengthy, fragmented text logs to understand the current status and history of a claim, delaying decision-making and customer response.

The Solution: The **Nexus Claims GenAI API** serves as a modern, scalable backend that automates the retrieval of claim status and leverages Generative AI (Amazon Bedrock) to instantly summarize complex claim history. This accelerates decision-making, reduces "time-to-insight" for adjusters, and improves customer satisfaction.

2. Solution Architecture

The following diagram illustrates the high-level architecture of the solution deployed on AWS.



3. Technology Choices & Rationale

AI/ML: Amazon Bedrock (The Specialized Choice)

- **Why:** A fully managed service for Generative AI that provides serverless access to high-performance foundation models.
- **Customer Benefit:**
 - **Instant Summarization:** We utilize the **Anthropic Claude** model to digest complex, unstructured text from claim notes (stored in S3) and produce a

concise, human-readable summary.

- **Operational Efficiency:** Unlike self-hosting LLMs on EC2/GPU instances, Bedrock requires **zero infrastructure management**. We simply make an API call.
- **Cost & Scale:** We pay only for the tokens processed, avoiding the high cost of idle GPU instances.

Compute: Amazon EKS (Elastic Kubernetes Service)

- **Why:** Industry-standard container orchestration.
- **Customer Benefit:** Provides a robust, scalable runtime for the Java application. It ensures high availability (pods are automatically restarted if they fail) and allows the business logic to scale independently of the database or AI components.

Database: Amazon DynamoDB

- **Why:** Serverless NoSQL database.
- **Customer Benefit:** Delivers single-digit millisecond latency for claim status lookups. It scales automatically to handle sudden spikes in claim queries without manual intervention or maintenance windows.

Storage: Amazon S3

- **Why:** Object storage for unstructured data.
- **Customer Benefit:** Stores the raw "adjuster notes" (text files) cost-effectively. S3 provides 99.999999999% durability, ensuring critical claim history is never lost.

Networking: Amazon API Gateway & Network Load Balancer

- **Why:** Secure and scalable entry point.
- **Customer Benefit:**
 - **API Gateway:** Acts as the "front door," managing traffic, security (throttling, potential auth), and providing a unified HTTP API endpoint.
 - **Network Load Balancer (NLB):** Provides high-performance, private connectivity between the API Gateway and the EKS cluster within the VPC. This ensures traffic remains secure and low-latency.

Infrastructure as Code: Terraform

- **Why:** Infrastructure provisioning.

- **Customer Benefit:** Enables "Environment as Code." The entire stack can be deployed, updated, or destroyed in minutes. This eliminates "configuration drift" and ensures the production environment matches development exactly.

4. Proof of Concept & Validation

API Output (Log Proof)

The following logs demonstrate the successful end-to-end execution of the API, including the GenAI summarization.

1. Retrieve Claim Status (DynamoDB Integration)

```
$ curl https://r7twk4cv09.execute-api.us-east-1.amazonaws.com/claims/claim-101

{
  "claimId": "claim-101",
  "status": "IN_PROGRESS",
  "customerId": "cust-555",
  "incidentDate": "2024-03-15",
  "description": "Car accident on highway"
}
```

2. Generate Claim Summary (Bedrock Integration)

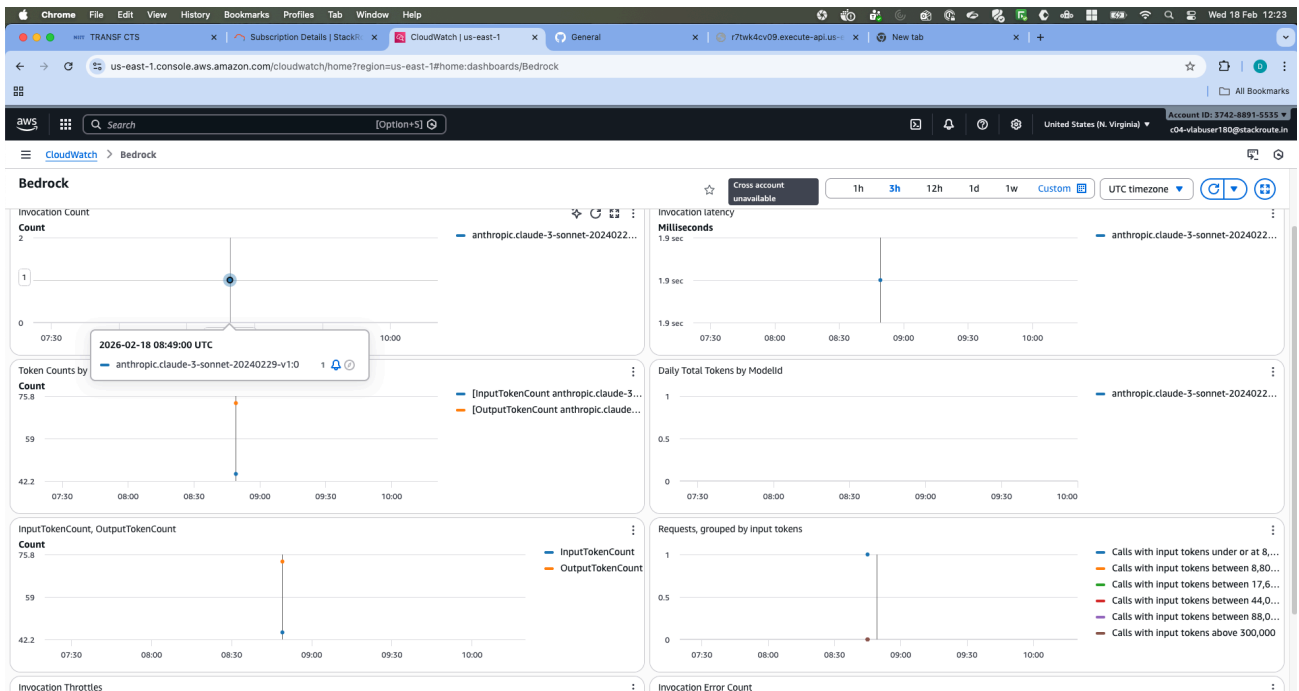
```
$ curl -X POST https://r7twk4cv09.execute-api.us-east-1.amazonaws.com/claims/claim-101/summary

{
  "notes_summary": "Here's a summary of the claim notes:\n\nThe customer reported a car accident on highway on 2024-03-15. The car was a 2018 Toyota Camry. The accident was caused by a sudden lane change by another vehicle. The car sustained significant damage to the front end. The driver was not injured. The car was towed to a nearby repair shop. The driver is currently at the shop waiting for the car to be repaired. The driver is asking for a claim to be filed for the car damage.",
  "claimId": "claim-101",
  "status": "IN_PROGRESS"
}
```

Infrastructure Screenshots

1. Amazon Bedrock (Metrics Proof)

Access path: AWS Console -> Amazon CloudWatch -> Metrics -> All metrics -> Bedrock -> Across all models *Goal:* Show the **InvocationCount** graph with data points, proving the API successfully called the model.

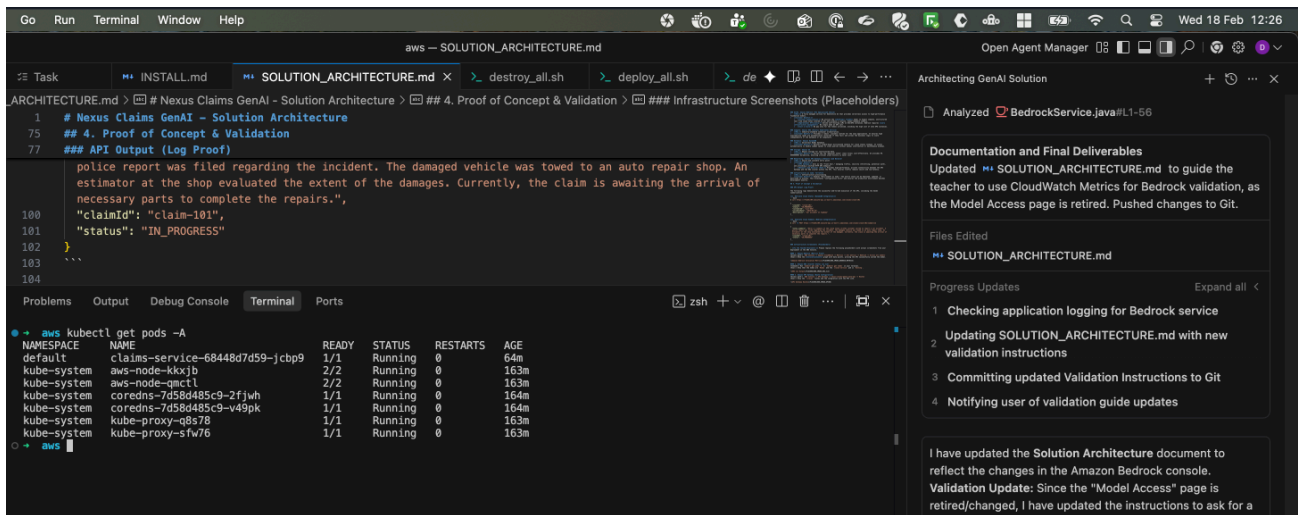


2. Amazon EKS (Cluster Status via CLI)

Command: Run `kubectl get pods -A` or `kubectl get nodes` in your terminal. *Goal:* Show that the nodes are `Ready` and the `claims-service` pod is `Running`.

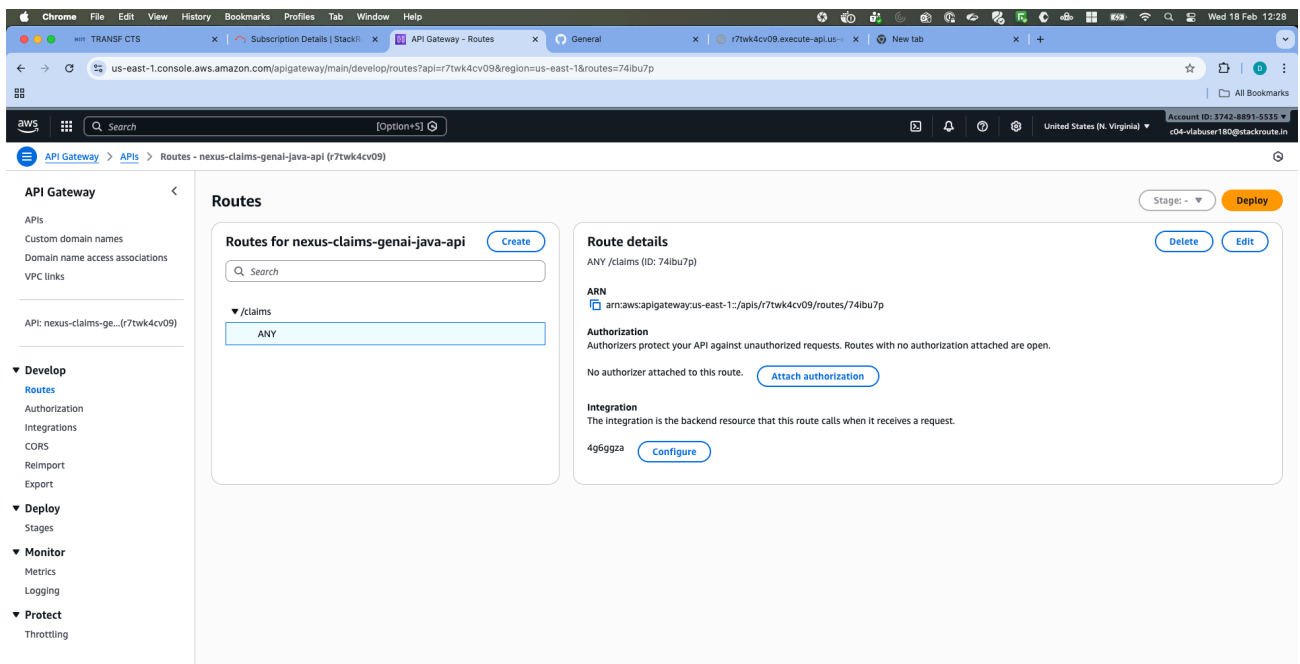
The screenshot displays the Amazon Elastic Kubernetes Service (EKS) console. The main dashboard shows the status of a cluster named `nexus-claims-genai-java-cluster`. The cluster is in the `Active` state. The table below shows the details of the cluster:

Cluster name	Status	Kubernetes version	Support period	Upgrade policy	Created	Provider
nexus-claims-genai-java-cluster	Active	1.34 Upgrade now	Standard support until December 2, 2026	Extended support	3 hours ago	EKS



3. Amazon API Gateway (Route Configuration)

Access path: AWS Console -> API Gateway -> nexus-claims-genai-java-api -> Routes
Goal: Show the `/claims` routes and the integration with the VPC Link.



4. AWS CodeBuild (Build History)

Access path: AWS Console -> CodeBuild -> Build projects -> nexus-claims-genai-java-build
Goal: Show a "Succeeded" build status.

The screenshot shows the AWS CodeBuild console for the project 'nexus-claims-genai-java-build'. The configuration section shows the source provider as GitHub, primary repository as 'danmariusp/nexus-claims-genai-java', and artifacts upload location as '-'. The build history section shows three build runs:

Build run	Status	Build number	Source version	Submitter	Duration	Completed
nexus-claims-genai-java-build:c7361536-154d-4abf-a805-4d7524895252	Succeeded	3	-	c04-vlabuser180@stackroute.in	1 minute 35 seconds	6 minutes ago
nexus-claims-genai-java-build:299f82b1-557d-4125-a3a8-77b7afcd0ed	Failed	2	-	c04-vlabuser180@stackroute.in	31 seconds	13 minutes ago
nexus-claims-genai-java-build:f528e556-72f5-4235-b2ef-81706a181736	Failed	1	-	c04-vlabuser180@stackroute.in	1 minute 47 seconds	20 minutes ago

5. Amazon ECR (Container Repository)

Access path: AWS Console → ECR → Repositories → nexus-claims-genai-java/claims-service **Goal:** Show the repository containing the latest image tag.

The screenshot shows the Amazon Elastic Container Registry (ECR) console for the repository 'nexus-claims-genai-java/claims-service'. The 'Images' tab is selected, showing a list of image tags:

Image tags	Type	Created at	Image size	Image digest	Last pulled at
f66dd48, latest	Image	18 February 2026, 11:21:34 (UTC+02)	128.38	sha256:c5752dd42bb9fb9...	18 February 2026, 11:21:38 (UTC+02)
-	Image Index	18 February 2026, 10:47:56 (UTC+02)	128.38	sha256:e3de31c4cba7e7ce...	18 February 2026, 10:48:38 (UTC+02)
-	Image	18 February 2026, 10:47:56 (UTC+02)	0.00	sha256:b2f13d57fd350ff...	18 February 2026, 10:48:33 (UTC+02)
-	Image	18 February 2026, 10:47:55 (UTC+02)	128.38	sha256:3790998712a7869...	18 February 2026, 10:48:38 (UTC+02)
-	Image Index	18 February 2026, 10:23:22 (UTC+02)	127.94	sha256:82b76f8d848f7582...	18 February 2026, 10:28:50 (UTC+02)
-	Image	18 February 2026, 10:23:22 (UTC+02)	127.94	sha256:54140f68c9c992de...	18 February 2026, 10:28:50 (UTC+02)
-	Image	18 February 2026, 10:23:22 (UTC+02)	0.00	sha256:ca25f21c5e3fd349...	-

6. Browser access

The screenshot shows a web browser displaying a JSON response from an API endpoint. The response is:

```
{
  "claimId": "claim-101",
  "status": "IN_PROGRESS",
  "customerId": "cust-555",
  "incidentDate": "2024-03-15",
  "description": "Car accident on highway"
}
```

5. API Usage

- `GET /claims/{id}` — Retrieve claim status from DynamoDB
- `POST /claims/{id}/summarize` — Generate AI summary of claim notes via Bedrock

6. Directory Structure

```
├── src/                # Application source code (Java Spring Boot) +
├── iac/                # Terraform infrastructure code
│   ├── bootstrap/     # IAM roles for deployment
│   └── main/           # Main infrastructure (EKS, VPC, DB, S3, API G
├── pipelines/          # CI/CD definitions (CodeBuild buildspec)
├── scripts/            # Deployment and data population scripts
├── mocks/              # Sample data for Claims and Notes
├── images/             # AWS Console screenshots (proof of deployment)
└── SOLUTION_ARCHITECTURE.md # This document
```

7. Deployment Guide

Prerequisites

Ensure you have the following tools installed and configured:

1. **AWS CLI (v2+)**: [Install Guide](#)
 - Run `aws configure` to set your credentials and default region (`us-east-1`).
2. **Terraform (v1.0+)**: [Install Guide](#)
3. **Docker**: [Install Guide](#) — ensure the daemon is running.
4. **Helm**: [Install Guide](#)
5. **kubectl**: [Install Guide](#)

Quick Start (Automated Deployment)

1. **Clone the Repository:**

```
git clone https://github.com/danmariusp/nexus-claims-genai-java.git
cd nexus-claims-genai-java
```

2. **Run the Deployment Script:**

```
./scripts/deploy_all.sh
```

This script will:

- Validate AWS credentials.
- Provision all infrastructure (IAM, VPC, EKS, DynamoDB, S3, API Gateway) via Terraform.
- Build the Java application using Docker.
- Push the Docker image to Amazon ECR.
- Deploy the application to EKS using Helm.
- Populate sample data into DynamoDB and S3.

3. **Verify Deployment:** The script outputs the **API Endpoint** at the end. Test it immediately:

```
curl <API_ENDPOINT>/claims/claim-101  
curl -X POST <API_ENDPOINT>/claims/claim-101/summarize
```

Manual Deployment Steps

If you prefer to deploy step-by-step or need to debug:

Step 1. Bootstrap Infrastructure

```
cd iac/bootstrap  
terraform init  
terraform apply -auto-approve  
ROLE_ARN=$(terraform output -raw builder_role_arn)
```

Step 2. Deploy Main Infrastructure

```
cd ../main  
terraform init  
terraform apply -auto-approve -var="builder_role_arn=$ROLE_ARN"
```

Step 3. Configure Kubernetes Access

```
aws eks update-kubeconfig --region us-east-1 --name nexus-claims-genai-jav
```

Step 4. Build and Push Docker Image

```
ECR_URL=$(terraform output -raw ecr_repository_url)
aws ecr get-login-password --region us-east-1 | docker login --username AI

docker build -t $ECR_URL:latest ../../src/claims-service
docker push $ECR_URL:latest
```

Step 5. Deploy Application

```
helm upgrade --install claims-service ../../src/helm/claims-service \
  --set image.repository=$ECR_URL \
  --set image.tag="latest"
```

Step 6. Populate Data

```
cd ../../
./scripts/populate_data.sh
```

8. Troubleshooting

Terraform Issues

Error	Solution
Error acquiring the state lock	Run <code>terraform force-unlock <LOCK_ID></code> if no other process is running.
NoCredentialProviders	Run <code>aws configure</code> to set valid access keys.

Application Issues

Symptom	Solution
503 Service Unavailable	The app may still be starting. Wait 1-2 minutes. Check: <code>kubectl get pods</code> and <code>kubectl logs -l app.kubernetes.io/name=claims-service</code>
GenAI Summary Fails	Ensure Anthropic Claude is accessible in Bedrock (<code>us-east-1</code>). First-time users may need to submit use case details. Check pod IAM permissions (IRSA).
Docker Push Access Denied	Ensure the CodeBuild role has <code>AmazonEC2ContainerRegistryPowerUser</code> policy.
EKS Unauthorized	Ensure the deploying user/role ARN is in EKS Access Entries (managed in <code>cicd.tf</code>).

9. Cleanup

To destroy all resources and avoid costs:

```
./scripts/destroy_all.sh
```